



```
In [1]: import re
```

```
In [2]: crick_scores=''Sachin scores 76 and Dravid scores 40 on Rohit scores 88
```

```
In [3]: name=re.findall('[A-Z][a-z]*',crick_scores)
```

```
In [4]: name
```

```
Out[4]: ['Sachin', 'Dravid', 'Rohit', 'Dhoni']
```

```
In [5]: score=re.findall('\d{2}',crick_scores)
```

```
In [6]: score
```

```
Out[6]: ['76', '40', '88', '99']
```

```
In [9]: str="My Institute Name is SkillCircle and SkillCircle is a very good Place"
```

```
In [10]: if re.search("SkillCircle",str):  
        print("Found")
```

```
Found
```

```
In [11]: match=re.findall('SkillCircle',str)  
        print(match)
```

```
['SkillCircle', 'SkillCircle']
```

```
In [12]: for i in re.finditer("SkillCircle",str):  
        index=i.span()  
        print(index)
```

```
(21, 32)
```

```
(37, 48)
```

```
In [13]: str2="Rat Cat Pat Mat Sat"  
        data=re.findall('[^RCPM]at',str2)
```

```
In [14]: data
```

```
Out[14]: ['Sat']
```

```
In [15]: data2=re.findall('[^P-R]at',str2)
```

```
In [16]: data2
```

```
Out[16]: ['Cat', 'Mat', 'Sat']
```

```
In [17]: str2="Rat Cat Pat Mat Sat"
```

```
In [18... reg=re.compile('[R]at')
str2=reg.sub("Lion",str2)
str2
```

Out[18]: 'Lion Cat Pat Mat Sat'

```
In [19... str4=''how are you
i am fine
thankyou'''
print(str4)
```

how are you
i am fine
thankyou

```
In [20... reg=re.compile("\n")
str4=reg.sub(" ",str4)
str4
```

Out[20]: 'how are you i am fine thankyou'

```
In [21... pattern="ABCD1623456678"
print("no of digits",len(re.findall('\d',pattern)))
```

no of digits 10

```
In [24... pattern="ABCD1623456678"
print(re.findall('\d',pattern))
```

['1', '6', '2', '3', '4', '5', '6', '6', '7', '8']

```
In [25... pattern="ABCD1623456678"
print(re.findall('\d{10}',pattern))
```

['1623456678']

```
In [26... print("no of alphabets",len(re.findall('\D',pattern)))      #--> Another
```

no of alphabets 4

```
In [27... m1="12 123 1234 12345 123456 1234567"
print("no of digits",len(re.findall('\d{5,7}',m1)))
```

no of digits 3

```
In [28... p1="111-1111-1111"
if re.search("\d{3}-\d{4}-\d{4}",p1):
    print("Valid")
```

Valid

```
In [29... # Remove websites links and URLs - \S matches all non-space characters and
url_text = "This is one type of url: https://www.wikipedia.com and the other"
print(re.findall("(http\S+)",url_text))
```

['https://www.wikipedia.com', 'http://www.wikipedia.com']

```
In [31... #Remove email addresses - Email addresses can be written in different form
#name_surname@domain.(suffix) or name-surname@domain.(suffix) etc.,

email_text = "Please contact us at priyanka-madiraju7445@company.com or p
print(re.findall("[\w\.-\_]+\@[\w\.-\_]+",email_text))

['priyanka-madiraju7445@company.com', 'priyanka.madiraju@company.com']
```

```
In [32... # Remove digits
numbers_text = "Phone Numbers are 33635353 and 9383777"
print(re.sub("(\\d+)", "", numbers_text))

Phone Numbers are  and
```

```
In [33... #Remove newline characters - sometimes the text has formatting like the n
newline_text="And I in going ,madam,weep o'er my father's death\\n\\n anew:
print(re.sub("(\\n+)", "", newline_text))

And I in going ,madam,weep o'er my father's death anew: I am now in war
d,evermore in subjection.
```

```
In [35... #Remove multiple spaces within text
url_text_output = "This is one type of url: and the other url is"
print(re.sub("(\\s){2,}", " ", url_text_output))

This is one type of url: and the other url is
```

```
In [36... #Remove special characters except for space
url_text_output = "This is one type of url: # ? and the other url is"
print(re.sub("[^a-zA-Z\\s]", "", url_text_output))

This is one type of url  and the other url is
```

```
In [37... # Detect and Remove HTML Tags
import re
tweet = '<p><html>follow this <b>website</b> for more details. </p></html>'
x=re.findall('<.*?>',tweet)
```

```
In [38... x

Out[38]: ['<p>', '<html>', '<b>', '</b>', '</p>', '</html>']
```

```
In [40... z=re.sub('<.*?>', "",tweet)
```

```
In [41... z

Out[41]: 'follow this website for more details. '
```

Word Lemmatization

- As a definition , lemmatization is to "remove inflectional endings only and

to return the base or dictionary form of word, which is known as the lemma."

- First, we create a raw text and tokenize it into tokens using the function `word_tokenize`.

```
In [42... import nltk
```

```
In [43... nltk.download('wordnet')
```

```
[nltk_data] Downloading package wordnet to  
[nltk_data] C:\Users\vipul\AppData\Roaming\nltk_data...  
[nltk_data] Package wordnet is already up-to-date!
```

```
Out[43]: True
```

```
In [45... import nltk  
nltk.download('punkt_tab')  
wnl = nltk.WordNetLemmatizer()  
text = "studies studying cries cry"  
tokenization = nltk.word_tokenize(text)  
for w in tokenization:  
    print("Lemma for {} is {}".format(w,wnl.lemmatize(w)))
```

```
[nltk_data] Downloading package punkt_tab to  
[nltk_data] C:\Users\vipul\AppData\Roaming\nltk_data...  
[nltk_data] Unzipping tokenizers\punkt_tab.zip.  
Lemma for studies is study  
Lemma for studying is studying  
Lemma for cries is cry  
Lemma for cry is cry
```

Stemming

```
In [50... from nltk.stem.porter import PorterStemmer #--> Pronunciation will same  
ps = PorterStemmer()  
text = "studies studying cries cry"  
tokenization = nltk.word_tokenize(text)  
for w in tokenization:  
    print("Stemming for {} is {}".format(w,ps.stem(w)))
```

```
Stemming for studies is studi  
Stemming for studying is studi  
Stemming for cries is cri  
Stemming for cry is cri
```

```
In [51... from nltk.stem.lancaster import LancasterStemmer #--> Pronunciation will  
lc = nltk.LancasterStemmer()  
text = "studies studying cries cry"  
tokenization = nltk.word_tokenize(text)  
for w in tokenization:  
    print("Stemming for {} is {}".format(w,lc.stem(w)))
```

```
Stemming for studies is study  
Stemming for studying is study  
Stemming for cries is cri  
Stemming for cry is cry
```

In []: